

JDBC SELECT Queries, Pagination and Sorting

Assumed table for Q1 to Q4:

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    name        VARCHAR(100),  
    price       DECIMAL(10,2),  
    category    VARCHAR(50)  
);
```

Change the URL, username and password to match your database.

Q1: Fetch all products and show them in a table

```
// ShowProducts.java  
// Fetches all products and prints name and price in a formatted console table.  
  
import java.sql.*;  
  
public class ShowProducts {  
    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";  
    private static final String USER = "root";  
    private static final String PASS = "root123";  
  
    public static void main(String[] args) {  
        String sql = "SELECT name, price FROM products";  
  
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
            Statement st = con.createStatement();  
            ResultSet rs = st.executeQuery(sql)) {  
  
            System.out.println("+-----+-----+");  
            System.out.printf("| %-24s | %10s |\n", "Product Name", "Price");  
            System.out.println("+-----+-----+");  
  
            while (rs.next()) {  
                System.out.printf("| %-24s | %10.2f |\n",  
                    rs.getString("name"), rs.getDouble("price"));  
            }  
            System.out.println("+-----+-----+");  
  
        } catch (SQLException e) {  
            System.out.println("Database error: " + e.getMessage());  
        }  
    }  
}
```

Sample output:

```
+-----+-----+  
| Product Name          |      Price |  
+-----+-----+  
| Wireless Mouse        |    599.00 |  
| USB Keyboard          |    899.50 |  
+-----+-----+
```

Q2: Filter by category (PreparedStatement)

```
// ProductsByCategory.java  
// Takes a category from the user and shows only that category's products.  
  
import java.sql.*;  
import java.util.Scanner;  
  
public class ProductsByCategory {  
    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";  
    private static final String USER = "root";  
    private static final String PASS = "root123";  
  
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
System.out.print("Enter category name: ");
String category = sc.nextLine();

String sql = "SELECT name, price FROM products WHERE category = ?";

try (Connection con = DriverManager.getConnection(URL, USER, PASS);
    PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setString(1, category);    // user input is passed as data, not SQL

    try (ResultSet rs = ps.executeQuery()) {
        System.out.println("+-----+-----+");
        System.out.printf("| %-24s | %10s |%n", "Product Name", "Price");
        System.out.println("+-----+-----+");

        boolean found = false;
        while (rs.next()) {          // ResultSet traversal
            found = true;
            System.out.printf("| %-24s | %10.2f |%n",
                rs.getString("name"), rs.getDouble("price"));
        }
        System.out.println("+-----+-----+");

        if (!found) {
            System.out.println("No products found in category: " + category);
        }
    }
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}

```

Q3: Pagination (5 products per page)

The page number is converted to an offset using (page - 1) * 5, and LIMIT ? OFFSET ? fetches only that page.

```

// ProductPagination.java
// Shows 5 products per page. The user enters the page number to view.

import java.sql.*;
import java.util.Scanner;

public class ProductPagination {
    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";
    private static final String USER = "root";
    private static final String PASS = "root123";
    private static final int PAGE_SIZE = 5;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {

            // Count total products to work out the total number of pages
            int totalProducts = 0;
            try (Statement st = con.createStatement();
                ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM products")) {
                if (rs.next()) totalProducts = rs.getInt(1);
            }
            int totalPages = (int) Math.ceil(totalProducts / (double) PAGE_SIZE);
            System.out.println("Total products: " + totalProducts + " | Total pages: " + totalPages);

            System.out.print("Enter page number: ");
            int page = sc.nextInt();

            if (page < 1 || page > totalPages) {
                System.out.println("Invalid page number.");
                return;
            }
        }
    }
}

```

```

int offset = (page - 1) * PAGE_SIZE;
String sql = "SELECT name, price FROM products ORDER BY product_id LIMIT ? OFFSET ?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, PAGE_SIZE);
    ps.setInt(2, offset);

    try (ResultSet rs = ps.executeQuery()) {
        System.out.println("\n--- Page " + page + " of " + totalPages + " ---");
        System.out.printf("%-25s %10s\n", "Product Name", "Price");
        System.out.println("-----");
        while (rs.next()) {
            System.out.printf("%-25s %10.2f\n",
                               rs.getString("name"), rs.getDouble("price"));
        }
    }
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}

```

Q4: Sorting by price or name

A ? placeholder cannot be used for a column name, so the program picks the column from a fixed list of allowed values. This keeps user input out of the SQL text and still prevents SQL injection.

```

// SortedProducts.java
// Lets the user choose to sort products by price (ascending) or by name.

import java.sql.*;
import java.util.Scanner;

public class SortedProducts {
    private static final String URL = "jdbc:mysql://localhost:3306/shop_db";
    private static final String USER = "root";
    private static final String PASS = "root123";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Sort products by:");
        System.out.println("1. Price (ascending)");
        System.out.println("2. Product name (A to Z)");
        System.out.print("Enter choice: ");
        int choice = sc.nextInt();

        // Whitelist: only these two fixed strings can ever reach the query
        String orderBy;
        if (choice == 1) {
            orderBy = "price ASC";
        } else if (choice == 2) {
            orderBy = "name ASC";
        } else {
            System.out.println("Invalid choice.");
            return;
        }

        String sql = "SELECT name, price FROM products ORDER BY " + orderBy;

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            System.out.println("\n" + String.format("%-25s %10s", "Product Name", "Price"));
            System.out.println("-----");
            while (rs.next()) {
                System.out.printf("%-25s %10.2f\n",
                                    rs.getString("name"), rs.getDouble("price"));
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}

```

```
}  
}
```

Q5: Playlist songs (AI-assisted code)

Step 1: Give this prompt to ChatGPT or Copilot:

"Write a JDBC SELECT query and Java code that fetches all songs for a given playlist_id from a MySQL table playlist_songs (playlist_id, song_id, title, artist) using PreparedStatement."

Step 2: Take a screenshot of the prompt and the generated code for your submission.

Step 3: Adapt it. Assumed table and generated query:

```
CREATE TABLE playlist_songs (  
    playlist_id INT,  
    song_id      INT,  
    title        VARCHAR(100),  
    artist       VARCHAR(100)  
);
```

```
SELECT title, artist FROM playlist_songs WHERE playlist_id = ?
```

Adapted Java code:

```
// PlaylistViewer.java  
// Displays all songs (title and artist) for a given playlist_id.  
  
import java.sql.*;  
import java.util.Scanner;  
  
public class PlaylistViewer {  
    private static final String URL = "jdbc:mysql://localhost:3306/spotify_clone";  
    private static final String USER = "root";  
    private static final String PASS = "root123";  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter playlist ID: ");  
        int playlistId = sc.nextInt();  
  
        String sql = "SELECT title, artist FROM playlist_songs WHERE playlist_id = ?";  
  
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
            PreparedStatement ps = con.prepareStatement(sql)) {  
  
            ps.setInt(1, playlistId);  
  
            try (ResultSet rs = ps.executeQuery()) {  
                System.out.println("\nPlaylist #" + playlistId);  
                System.out.printf("%-30s %-25s\n", "Song Title", "Artist");  
                System.out.println("-----");  
  
                int count = 0;  
                while (rs.next()) {  
                    System.out.printf("%-30s %-25s\n",  
                                        rs.getString("title"), rs.getString("artist"));  
                    count++;  
                }  
                if (count == 0) {  
                    System.out.println("No songs found for this playlist.");  
                }  
            } catch (SQLException e) {  
                System.out.println("Database error: " + e.getMessage());  
            }  
        }  
    }  
}
```

Sample output:

```
Playlist #1  
Song Title          Artist
```

```
-----  
Blinding Lights      The Weeknd  
Levitating           Dua Lipa
```

Adaptation notes to write in your assignment:

- Changed the output to show only the title and artist columns in aligned columns using printf.
- Added a Scanner so the playlist ID is entered by the user instead of being hard-coded.
- Added a "No songs found" message for empty playlists.
- Used try-with-resources so the connection, statement and result set close automatically.